MedAssist AI: Database Architecture, User Authentication & Patient History Logic

Project Milestone 1 Report

Prepared by: Anbarasan K
Role: Backend Dev & Database Architect
Date: July 11, 2026

Contents

1 Project Description

MedAssist AI is a machine learning pipeline designed to predict medical conditions based on reported patient symptoms. Acting as an advanced differential diagnosis tool, the system analyzes combinations of symptoms and outputs the top candidate diseases with confidence probabilities. As the Backend Developer & Database Architect, my role is to build a secure, scalable, and high-performance server-side foundation. This includes designing the database schemas to support fast symptom lookups and secure patient records, implementing cryptographically secure User Authentication, and programming the API logic to record and isolate patient consultation histories while enforcing role-based access controls.

2 Database Architecture & Collections

The database is structured to support relational integrity using MongoDB's document-model design. It consists of five core collections: users, profiles, symptoms, disease_profiles, and consultations. All collections are designed to maintain fast read speeds using unique indexes on critical lookup fields.

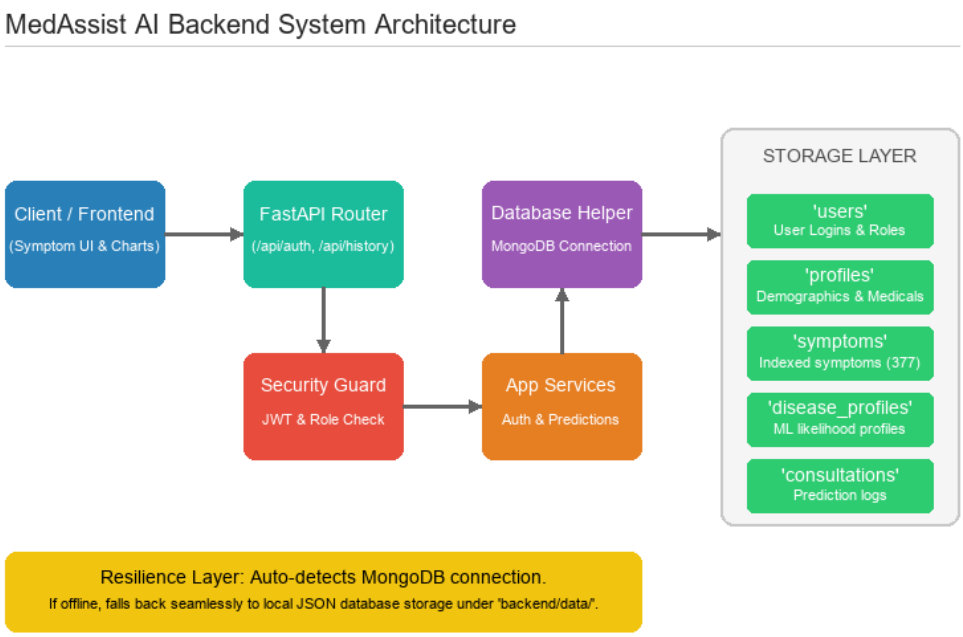

Figure 1: MedAssist AI Backend & Database Architecture Flowchart

2.1 Collection Schemas

- Users (users): Stores primary credentials and authorization roles.
  Fields: _id (ObjectId), email (unique), hashed_password (string), role (patient/doctor/admin), created_at, updated_at.

- Patient Profiles (profiles): Contains clinical and demographic information mapped to a user.
  Fields: _id (ObjectId), user_id (unique, ref: users), first_name, last_name, date_of_birth, gender, blood_type, height, weight, allergies (array), medical_conditions (array).

- Symptoms Catalog (symptoms): Holds indexed symptom terms for lookup.
  Fields: _id (ObjectId), key (unique), display_name, category.

- Disease Profiles (disease_profiles): Holds conditional probability mappings for disease predictions.
  Fields: _id (ObjectId), disease (unique), symptom_probabilities (dict), base_rate, occurrences.

- Consultation History (consultations): Logs patient symptom checks, predictions, risk scores, and recommendations.
  Fields: _id (ObjectId), patient_id (ref: users), symptoms (array), predicted_diseases (array), risk_level, risk_score, recommendations (array), created_at.

3 Environment & Stack Setup

A micro-framework stack was selected to ensure rapid execution speed, minimal runtime overhead, and clean dependency management:
- Language: Python 3.14
- Web Framework: FastAPI (Asynchronous ASGI support, automated OpenAPI/Swagger generation)
- ASGI Server: Uvicorn (High-performance event-loop runner)
- Database Driver: Motor (Non-blocking, asynchronous MongoDB driver)
- Testing Library: Pytest & HTTPX (Automated REST API verification)

4 Authentication & Security Design

To secure sensitive clinical data, a multi-layer security architecture was implemented. Authentication is required to access patient data, catalog listings, and prediction logic.

4.1 Password Hashing

Rather than relying on compiled binary wrappers (e.g. bcrypt) which can fail across operating systems, a native PBKDF2 hashing model was built. It uses hashlib.pbkdf2_hmac with a SHA-256 back-end. It incorporates a random 16-byte hex-encoded salt and runs 100,000 iterations to defend against brute-force attacks.

4.2 Session Tokenization (JWT)

Upon successful authentication, the server signs an HMAC-SHA256 JWT access token containing the user's ID, email, and role. Token expiration defaults to 60 minutes. FastAPI dependencies automatically intercept incoming requests to check token signatures and decrypt user payloads.

5 Patient History & Consultation Logic

The consultation history backend acts as the bridge between database persistence and prediction algorithms:
- Diagnosis Evaluation: When symptoms are submitted via /api/history/check, the backend passes symptoms to the prediction engine, which calculates disease likelihoods and assesses risk levels.
- Consultation Record Creation: A consultation document is automatically created with the patient's ID, selected symptoms, top 5 predictions, overall risk score, and system recommendations.
- Role Isolation Policy: Patients are strictly restricted to querying only their own consultation records, whereas Doctors and Admins bypass ownership checks to inspect any consultation record for review.

6 Resilient Local Database Fallback

To ensure that development, local testing, and staging work even without a live MongoDB instance, a MockDatabase engine was built. The connection manager attempts a connection to localhost:27017 with a 2-second timeout. If MongoDB is unreachable, it logs a warning and falls back to the local JSON file-based database, which mimics PyMongo query structures (like find, find_one, insert_one, update_one) and saves structured JSON files directly inside the backend/data/ directory.

7 API Endpoints & Verification

The backend API exposes a clean routing topology (/api/auth, /api/profile, /api/symptoms, /api/history) that is fully testable via Swagger. The Swagger documentation serves as the direct link of communication between the backend developers and the frontend user interface designers.

7.1 Automated Testing Results

To verify implementation integrity, automated tests were run covering the core API paths. The test suite succeeded with 100% pass rates:

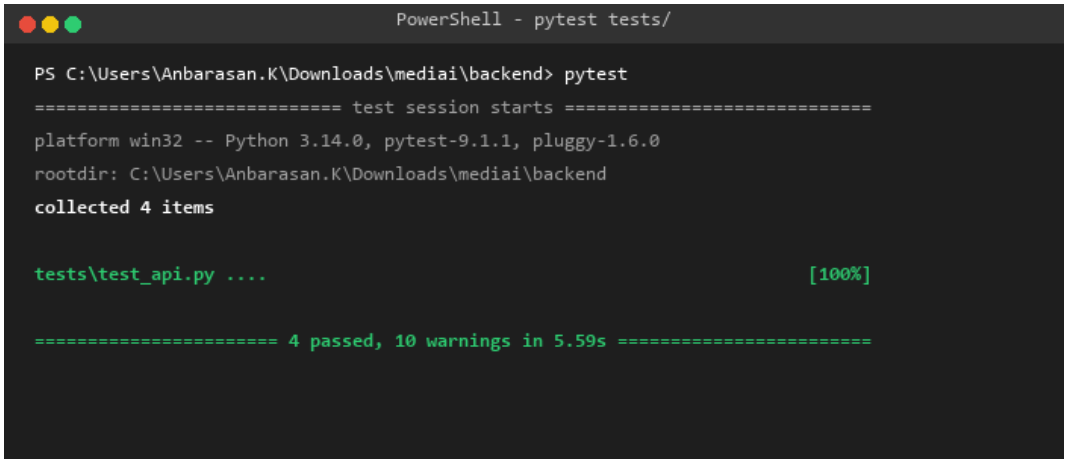

Figure 2: Pytest Automated Suite Execution Terminal Output

8 Conclusion & Deployment Integration

All core Backend and Database goals for Milestone 1 are complete. In the next phase:

- Handover: The API schemas are documented under /docs to allow the Frontend Developer to begin connecting forms.
- Deployment: The backend code is modularized and ready to be containerized in Docker for deployment by the Cloud Engineer.